

Streams and Change Data Capture

DATA PIPELINE AUTOMATION IN SNOWFLAKE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

Processing Only What's Changed

CDC: Change Data Capture



RUN 1) FULL TABLE SCAN
10 MILLION ROWS PROCESSED, 500 CHANGED



RUN 2) CHANGE DATA CAPTURE
ONLY 500 ROWS CHANGED

20,000X MORE WORK WITHOUT CDC

Streams

What a stream does

- Tracks every INSERT, UPDATE, and DELETE on a source table

PRODUCT	QUANTITY	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
APPLE	10	DELETE	FALSE	ec39f20dfa82444027d808246e412fc04615a8e3
ORANGE	30	INSERT	TRUE	70757a8215789624f74fda88faa544dba6ef82c4
ORANGE	50	DELETE	TRUE	70757a8215789624f74fda88faa544dba6ef82c4
PEACH	35	INSERT	FALSE	6b77a44ba5dcf099648bacd1cc51965298ee9cb2

- Maintains a running change log — no data duplication
- Once consumed, the **offset advances**; next read starts fresh

¹ * Snowflake Learning Resource

Stream Types

Stream Type	Captures	Best For
Standard	All table types and views & all DML changes - tracks inserts, updates, deletes	Tables where any row can change (e.g. shipments)
Append-only	All table types and views, except external tables - tracks row inserts only	Insert-once tables (e.g. delivery events) - more efficient
Insert-only	Externally managed Apache Iceberg and external tables - tracks row inserts only	External tables

Directory tables surface file metadata for a stage (name, size, last modified timestamp)

Creating a Stream

Standard stream on the shipments table

```
CREATE STREAM shipments_stream  
ON TABLE logistics.shipments;
```

Append-only stream on the delivery events table

```
CREATE STREAM delivery_events_stream  
ON TABLE logistics.delivery_events  
APPEND_ONLY = TRUE;
```

Stream Metadata Columns

```
SELECT product, quantity, METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID
FROM shipments_stream;
```

- **METADATA\$ACTION** : INSERT or DELETE
- **METADATA\$ISUPDATE** : TRUE when part of an update pair
- **METADATA\$ROW_ID** : Unique physical row identifier
- Updates appear as a **DELETE + INSERT** pair, both flagged with **METADATA\$ISUPDATE = TRUE**

PRODUCT	QUANTITY	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
APPLE	10	DELETE	FALSE	ec39f20dfa82444027d808246e412fc04615a8e3
ORANGE	30	INSERT	TRUE	70757a8215789624f74fda88faa544dba6ef82c4
ORANGE	50	DELETE	TRUE	70757a8215789624f74fda88faa544dba6ef82c4
PEACH	35	INSERT	FALSE	6b77a44ba5dcf099648bacd1cc51965298ee9cb2

↑
represents an update

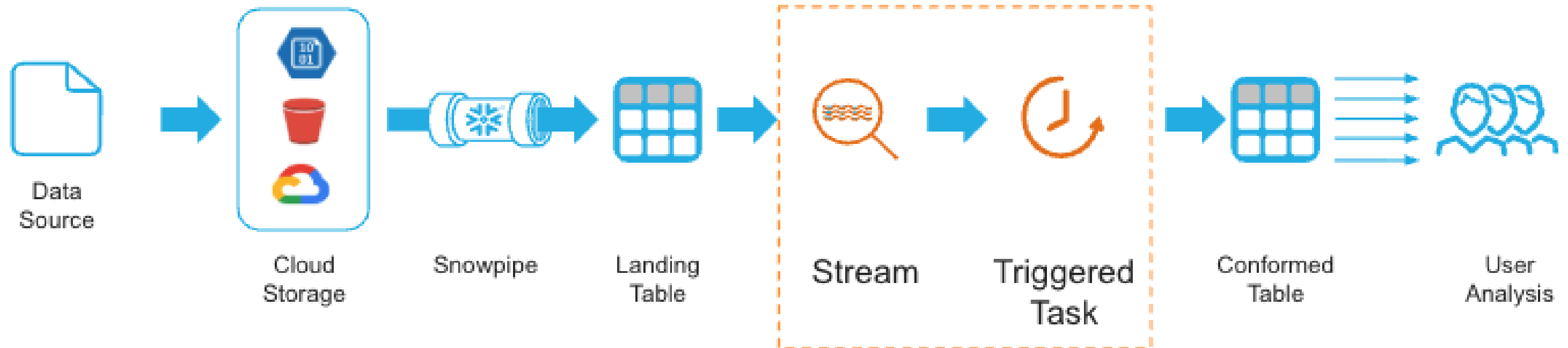
← stays the same

The Stream Offset



Streams in a Pipeline Overview

- Streams pair with tasks — Snowflake objects that run SQL on a schedule
- Task reads only changed rows; with 10M rows: 2 seconds vs 2 minutes



¹ * Snowflake Learning Resource

Streams in a Pipeline Query

- Streams pair with tasks - Snowflake objects that run SQL on a schedule
- Task reads only changed rows; with 10M rows: 2 seconds vs 2 minutes

```
CREATE TASK logistics.sync_shipments
  WAREHOUSE = compute_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('logistics.staging_shipments_stream')
AS
  INSERT INTO logistics.shipments
  SELECT shipment_id, region, carrier, delivery_days
  FROM logistics.staging_shipments_stream
  WHERE METADATA$ACTION = 'INSERT';
```

Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

Tasks and DAG Orchestration

DATA PIPELINE AUTOMATION IN SNOWFLAKE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

Solution For Manual Workloads: Tasks

- Team runs a SQL script manually each morning
- One missed run - business loses visibility
- **Snowflake tasks** automate this entirely

Example:

```
-- Today's manual routine (fragile)  
CALL logistics.refresh_ops_dashboard();
```

What Do We Need To Know About Tasks

- Tasks run SQL on a schedule — no external CRON required
- Execute SQL statements or call stored procedures
- Compute and scheduling live natively in Snowflake

```
CREATE TASK logistics.refresh_dashboard
  WAREHOUSE = harbr_wh
  SCHEDULE = 'USING CRON 0 6 * * * UTC'
AS CALL logistics.refresh_ops_dashboard();
```

Creating a Standalone Task

```
CREATE OR REPLACE TASK transform_delivery_summary
  WAREHOUSE = harbr_wh
  SCHEDULE = 'USING CRON 5 * * * * UTC'
  AS INSERT INTO delivery_summary

SELECT shipment_id, status, updated_at
FROM delivery_events
WHERE processed = FALSE;
```

Warehouse-based vs Serverless

Warehouse-based

- Uses a named virtual warehouse
- 60-second minimum billing per run

```
CREATE TASK my_task
  WAREHOUSE = harbr_wh -- named warehouse
  SCHEDULE = '5 MINUTE'
AS INSERT INTO ...;
```

Serverless

- Omit the `WAREHOUSE` clause
- Billed per second consumed; no idle cost

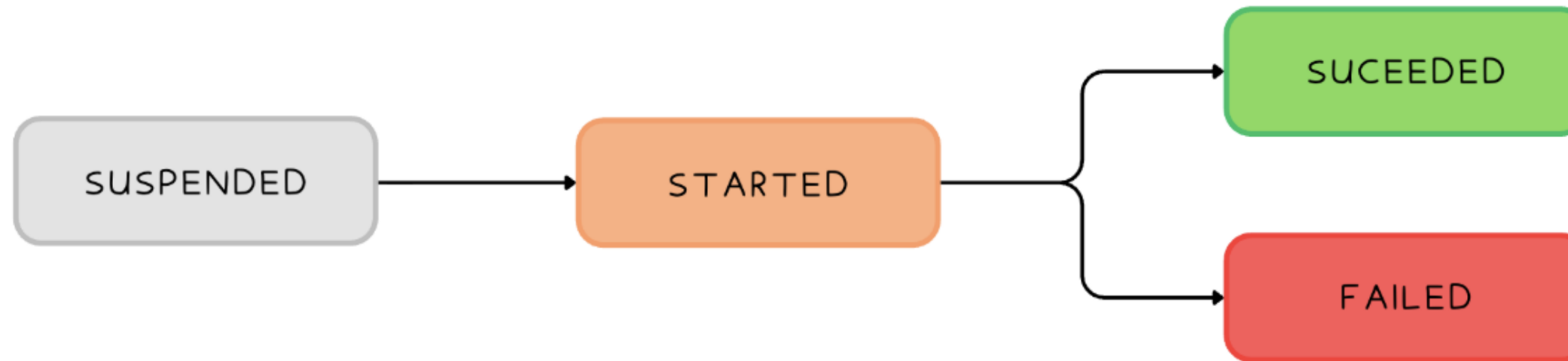
```
CREATE TASK my_serverless_task
  -- no WAREHOUSE clause
  SCHEDULE = '5 MINUTE'
AS INSERT INTO ...;
```


DAAG-based Task Orchestration



- DAGs: Directed Acyclic Graph
- Root task holds the CRON schedule
- Child tasks declare their predecessor with `AFTER`
- Entire chain is centralized
- DAGs control when and in what order things run

Managing Task States



-- Activate a task

```
ALTER TASK mytask RESUME;
```

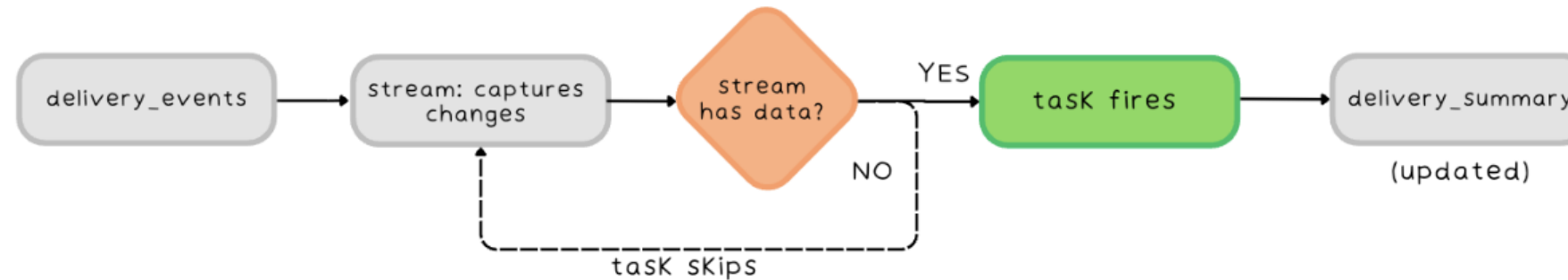
-- Pause a task

```
ALTER TASK mytask SUSPEND;
```

-- Execute a task

```
EXECUTE TASK mytask SUSPEND;
```

Tasks and Streams Combined



```
CREATE OR REPLACE TASK logistics.process_delivery_events
  WAREHOUSE = compute_wh
  SCHEDULE = '5 minute'
  WHEN SYSTEM$STREAM_HAS_DATA('logistics.delivery_events_stream')
AS
INSERT INTO logistics.processed_events
SELECT * FROM logistics.delivery_events_stream;
```

Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

Dynamic Tables

DATA PIPELINE AUTOMATION IN SNOWFLAKE

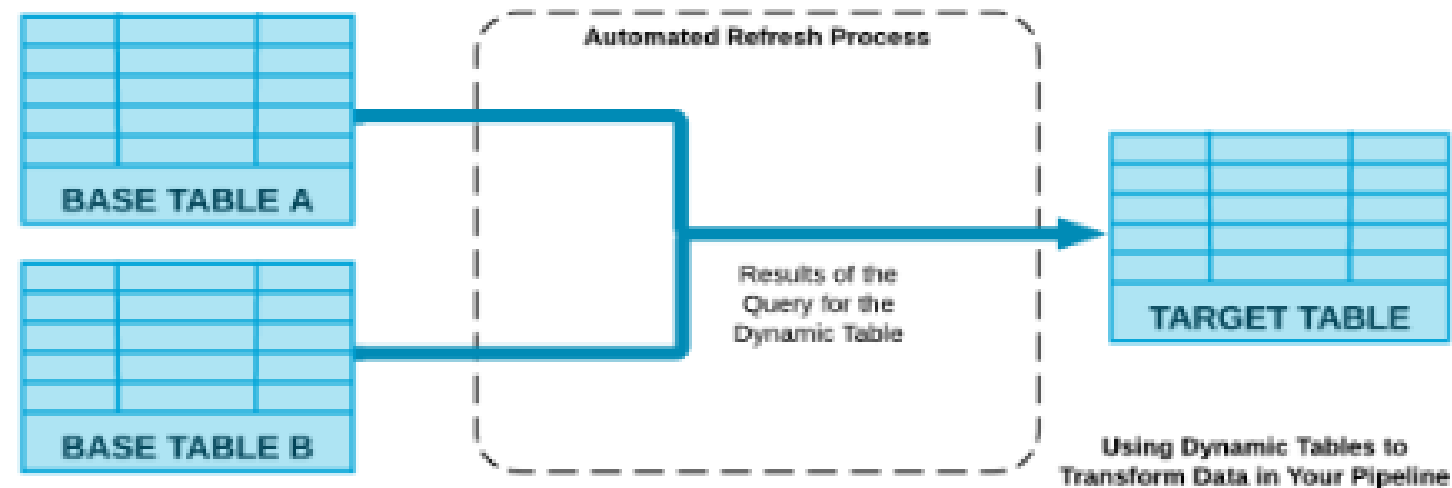


Emily Melhuish

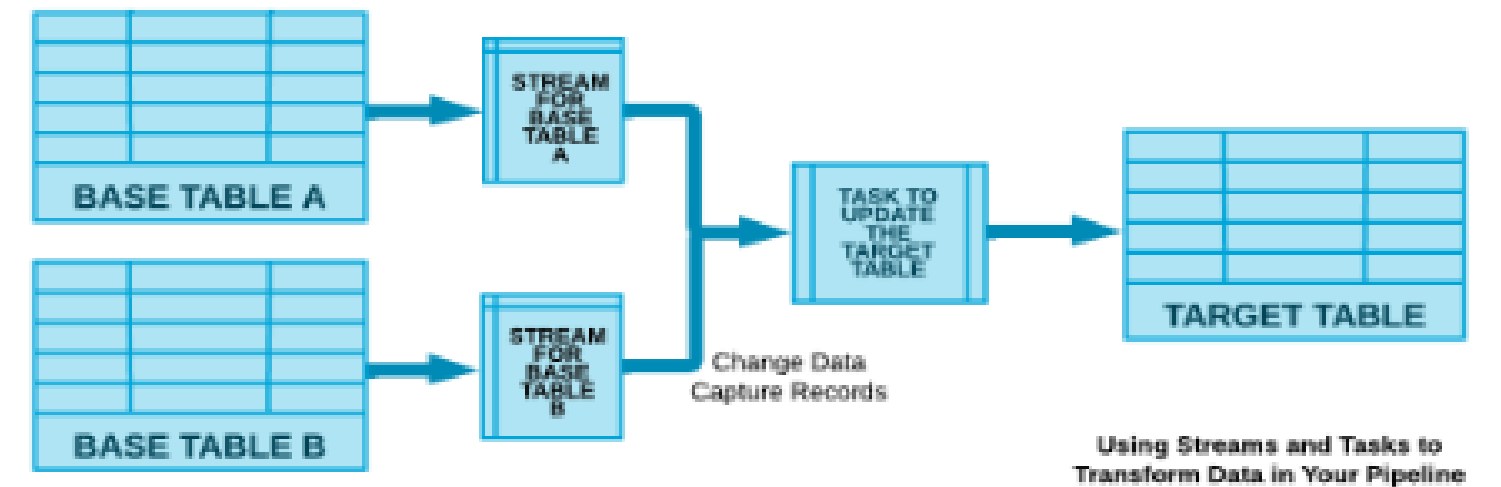
Technical Curriculum Developer,
Snowflake

Dynamic Tables vs Streams and Tasks

Dynamic Tables:



Streams and tasks:



- Simplify data pipeline transformation using SQL declarative statements
- Automatic refresh process
- Snowflake detects, schedules and merges

¹ * Snowflake Learning Material

What are Dynamic Tables?

Declarative pipeline automation

- Contents defined by a `SELECT` query — describe the result, Snowflake manages the refresh
- No stream configuration, no tasks to schedule

```
CREATE DYNAMIC TABLE logistics.delivery_summary
  TARGET_LAG = '1 hour'
  WAREHOUSE = harbr_wh
AS
  SELECT region, COUNT(*) AS shipments,
         AVG(delivery_days) AS avg_days
  FROM logistics.shipments GROUP BY region;
```

TARGET_LAG: Reduce Staleness

The freshness contract between you and Snowflake

TARGET_LAG value	Effect
'5 minutes'	Table is never more than 5 minutes stale - refreshes frequently
'1 hour'	Refreshes less often - lower compute consumption
DOWNSTREAM	Infer lag from downstream dependents - ideal for chained tables

Full vs Incremental Refresh

Incremental refresh

- Tracks exactly which source rows changed
- Refreshes only affected rows — faster, cheaper at scale

```
-- Incremental-friendly:  
SELECT region, COUNT(*) AS shipments  
FROM logistics.shipments  
GROUP BY region;
```

Full refresh

- Reprocesses the entire query
- Required for certain aggregations and set operations

Chaining Dynamic Tables



- Each transformation is a separate dynamic table with its own query
- Refreshes propagate automatically through the chain
- `DOWNSTREAM` lag defers to what the next table needs - no over-refreshing

Dynamic Tables vs Streams and Tasks

Dynamic tables: declarative

- Describe the output; Snowflake manages refresh, easy to manage
- Best for analytics/reporting layers

```
-- Declare the result for Snowflake
CREATE DYNAMIC TABLE logistics.summary
  TARGET_LAG = '1 hour'
  WAREHOUSE = harbr_wh
AS SELECT region, COUNT(*)
FROM shipments
GROUP BY 1;
```

Streams + Tasks: imperative

- Control what runs and when
- Best for branching complex custom logic and operational pipelines

```
-- You control every step
CREATE TASK process_events
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA
    ('events_stream')
AS CALL logistics.process_new_events();
```

DYNAMIC_TABLE_REFRESH_HISTORY

```
SELECT name, state, refresh_start_time, refresh_end_time
FROM TABLE (
  INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY (
    NAME_PREFIX => 'HARBR_DB.DELIVERY_SUMMARY.', ERROR_ONLY => TRUE
  )
)
ORDER BY name, data_timestamp;
```

NAME	TRIGGER	STATE	START	END
DELIVERY_SUMMARY	SCHEDULED	SUCCEEDED	2026-03-01 05:00:00	2024-03-01 05:00:08
DELIVERY_SUMMARY	SCHEDULED	FAILED	2026-03-01 04:00:00	2024-03-01 04:00:03

Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

External and Iceberg Tables

DATA PIPELINE AUTOMATION IN SNOWFLAKE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

What is an External Table?

Use case:

- Harbr receives files from dozens of partners daily
- Loading them into Snowflake doesn't make sense

External Tables

- Data stays in S3, GCS, or Azure Blob — never moves into Snowflake
- Snowflake stores metadata: file paths, schemas
- Reads files at query time



Creating an External Table

```
CREATE EXTERNAL TABLE logistics.supplier_inventory  
  WITH LOCATION = @harbr_stage/inventory/  
  FILE_FORMAT = (TYPE = 'PARQUET')  
  AUTO_REFRESH = TRUE;
```


When to Use External Tables

- **Data is owned by a partner** — query without taking on data management
- **Files are large, queried infrequently** — avoid loading data accessed once a month
- **Data cannot move for compliance** — files must stay in their original location
- **Exploring before committing** — read raw files before deciding what to load

```
-- Query partner inventory
SELECT supplier_id,
        COUNT(*)      AS file_count,
        SUM(quantity) AS total_units
FROM logistics.supplier_inventory
WHERE delivery_date >= '2024-01-01'
GROUP BY supplier_id;
```


External Tables vs Loading Data

External tables

- Zero Snowflake storage cost
- Always reflects current cloud storage state
- Best for infrequently queried, archived data

```
-- Query directly, no load needed
SELECT * FROM
logistics.supplier_inventory
WHERE supplier_id = 'SUP-042';
```

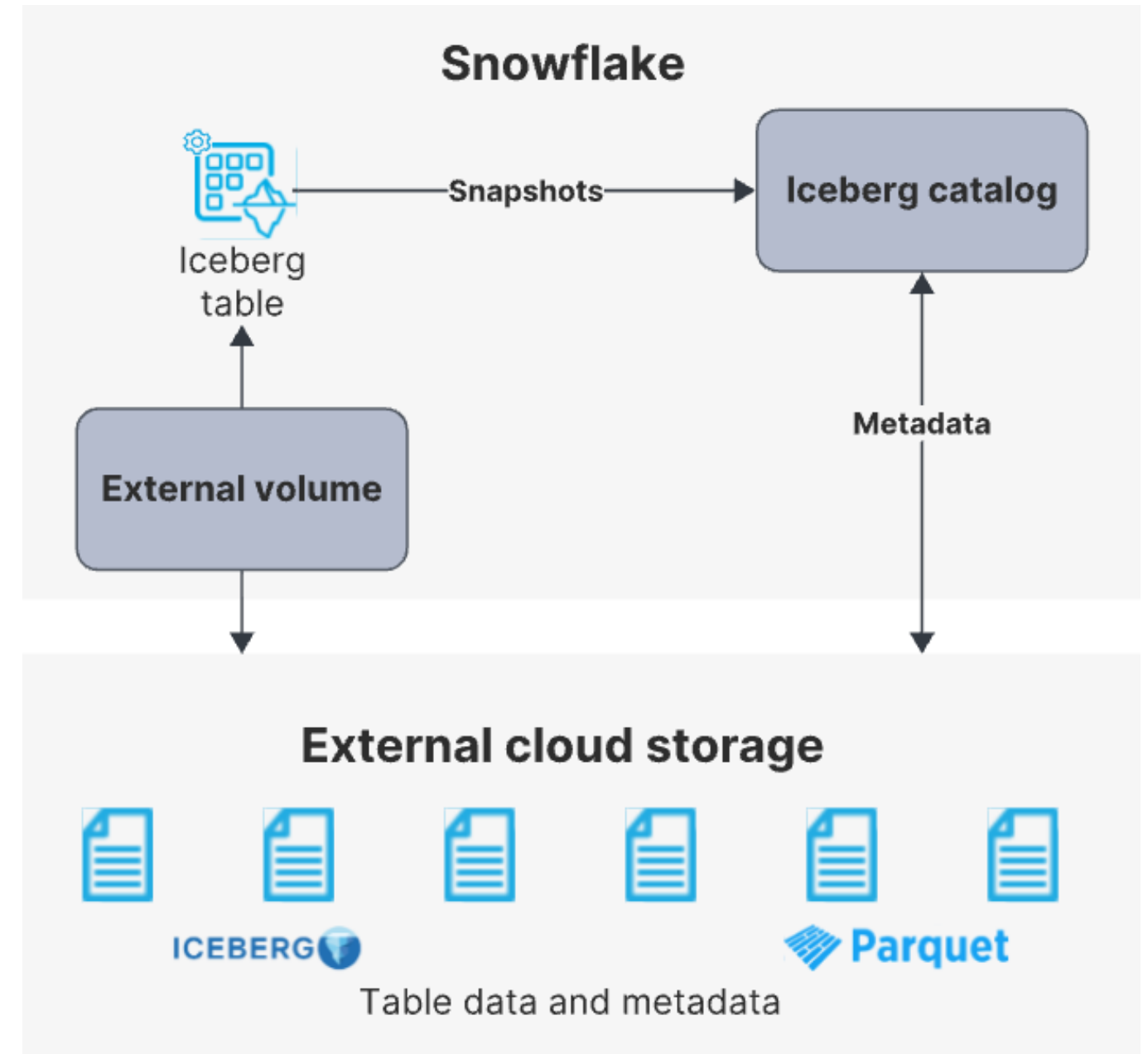
Loading with COPY INTO

- Full Snowflake query performance
- Best for frequently queried, live data

```
-- Load once, query fast
COPY INTO logistics.delivery_events
FROM @harbr_stage/events/;
```

Apache Iceberg Tables

- How do we ensure we read data correctly from S3, Azure Blob or GCS?
 - Apache
- **Apache** = Open table format: data not locked to any engine
 - Parquet files + metadata layer (time travel, schema history, partitions)
 - database-grade reliability



¹ * Snowflake learning material

Snowflake-managed vs Externally-managed

Snowflake-managed

- Snowflake owns and writes the Iceberg metadata
- Full read-write access from SQL

```
CREATE ICEBERG TABLE
logistics.shipments
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'harbr_s3_vol'
  BASE_LOCATION = 'iceberg/shipments/';
```

Externally-managed

- Outside catalog owns metadata - AWS Glue, Apache Polaris

```
CREATE ICEBERG TABLE
logistics.partner_data
  CATALOG = 'glue_catalog'
  EXTERNAL_VOLUME = 'harbr_s3_vol'
  CATALOG_TABLE_NAME =
    'partner.deliveries';
```

Let's Practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE